ReactFlow Canvas

Frontend Intern Assignment — Complete Implementation Guide

Tech Stack

React + Vite	TypeScript (strict)	ReactFlow (xyflow)	shadcn/ui	TanStack Query	Zustand	MSW

Objective Overview	
Goal	Build a responsive App Graph Builder UI matching the provided screenshot.
Key areas	Layout · ReactFlow · Node Inspector · TanStack Query · Zustand · TypeScript
Deliverable	GitHub repo with source code + README (setup, decisions, limitations)
Optional deploy	Vercel or Cloudflare Pages

1. Project Folder Structure

Create the Vite project with `npm create vite@latest app-graph-builder -- --template react-ts`. Then organise the `src/` folder as below — one concern per folder.

`src/`	Root source directory
`src/main.tsx`	App entry-point (QueryClient + providers)
`src/App.tsx`	Root layout shell
`src/store/`	Zustand store slices
`src/store/useAppStore.ts`	selectedAppId · selectedNodeId · isMobilePanelOpen · activeInspectorTab
`src/mocks/`	MSW mock handlers
`src/mocks/handlers.ts`	GET /apps & GET /apps/:id/graph
`src/mocks/browser.ts`	MSW browser worker setup
`src/api/`	TanStack Query fetch functions
`src/api/appsApi.ts`	fetchApps() — calls /apps
`src/api/graphApi.ts`	fetchGraph(appId) — calls /apps/:id/graph
`src/hooks/`	Custom TanStack Query hooks
`src/hooks/useApps.ts`	useQuery wrapper for apps list
`src/hooks/useGraph.ts`	useQuery wrapper for graph (refetches on appId change)
`src/components/layout/`	Shell components

`src/components/layout/TopBar.tsx`	Brand + Fit-view button + placeholder actions
`src/components/layout/LeftRail.tsx`	Icon-style vertical nav (static)
`src/components/layout/RightPanel.tsx`	App list + Node Inspector; becomes drawer on mobile
`src/components/canvas/`	ReactFlow components
`src/components/canvas/AppCanvas.tsx`	ReactFlow provider + nodes + edges + controls
`src/components/canvas/ServiceNode.tsx`	Custom node card (status badge + cost badge)
`src/components/inspector/`	Node inspector panel
`src/components/inspector/NodeInspector.tsx`	Tabs: Config tab + Runtime tab
`src/components/inspector/SliderInput.tsx`	Synced slider ↔ numeric input component
`src/components/apps/`	App selector components
`src/components/apps/AppList.tsx`	Searchable list from /apps
`src/components/apps/AppListItem.tsx`	Single app row
`src/types/`	Shared TypeScript types
`src/types/index.ts`	App · GraphData · NodeData · EdgeData types
`public/mockServiceWorker.js`	MSW service worker (auto-generated)
`tsconfig.json`	strict: true + path aliases
`.eslintrc.cjs`	ESLint for React + TS
`vite.config.ts`	Vite config (aliases, plugins)

---

2. Layout Breakdown

The UI is divided into four zones rendered by `App.tsx`. Use CSS Grid or Flexbox for the outer shell.

Zone	Component	Responsibility
Top Bar	TopBar.tsx	Brand logo/title · Fit-view button · theme/action icons
Left Rail	LeftRail.tsx	Vertical icon nav (GitHub, DB, Redis … static icons from screenshot)
Center Canvas	AppCanvas.tsx	ReactFlow with dotted background — takes all remaining space
Right Panel	RightPanel.tsx	App selector list (top) + Node Inspector (bottom when node selected)

Responsive rule: When `window.width < 768px`, the Right Panel hides and slides in as a drawer. The open/close state lives in Zustand (`isMobilePanelOpen`). A hamburger icon in the Top Bar toggles it.

---

3. ReactFlow Canvas

Setup

Wrap `AppCanvas.tsx` with `<ReactFlowProvider>`. Import `Background`, `Controls`, `MiniMap` from `@xyflow/react`.

Use `Background variant='dots'` for the dotted canvas seen in the screenshot.

Node & Edge data shape

Nodes come from `GET /apps/:id/graph`. Each node carries a `data` object:

```
{ id, label, status: 'Healthy'|'Degraded'|'Down', cost: '$0.03/HR', sliderValue: 0 }
```

Required interactions

- **Drag** — enabled by default in ReactFlow.
- **Select** — on node click, call `setSelectedNodeId(node.id)` in Zustand.
- **Delete** — listen to `onKeyDown` (Delete/Backspace) and call `setNodes(nodes.filter(n => n.id !== selectedNodeId))`.
- **Zoom / Pan** — ReactFlow default, no extra code needed.
- **Fit View** — call `reactFlowInstance.fitView()` from the Top Bar button.

Custom ServiceNode card — matches screenshot

- Logo icon + service name on top-left
- Cost badge (e.g. '$0.03/HR') + settings gear icon on top-right
- Metric tabs: CPU · Memory · Disk · Region
- Coloured slider bar + numeric value
- AWS logo bottom-right
- Status pill at bottom-left (Success = green, Error = red, Degraded = yellow)

---

4. Node Inspector (Right Panel)

Shown in the right panel whenever `selectedNodeId !== null`. Built entirely with **shadcn/ui** primitives.

Part	shadcn/ui Component	Behaviour
Status Pill	Badge	Colour changes: green=Healthy, yellow=Degraded, red=Down
Node Name	Input	Editable — updates node data in ReactFlow state on blur/enter
Tabs	Tabs / TabsList / TabsContent	Min 2 tabs: Config + Runtime (Zustand: activeInspectorTab)
Slider	Slider	0–100, synced with numeric Input both ways
Numeric Input	Input (type=number)	Synced with slider, persisted to node data
Description	Textarea (optional)	Free-text notes for the node

Sync pattern for Slider + Input: Keep `sliderValue` in the node's `data` object. When slider changes → update node data → input reads from node data. When input changes → validate 0–100 → update node data → slider reads from node data.

---

5. TanStack Query + Mock API (MSW)

MSW Handlers to create (`src/mocks/handlers.ts`)

- `GET /apps` — Returns array of { id, name, icon, color }. Add ~300ms setTimeout delay.
- `GET /apps/:appId/graph` — Returns { nodes: [...], edges: [...] }. Add ~500ms delay. Can randomly throw 500 to simulate error.

Sample app list data (hardcoded in handler)

```
supertokens-golang · supertokens-java · supertokens-python · supertokens-ruby · supertokens-go
```

Sample graph data per app

4 nodes matching screenshot: Postgres · Redis · MongoDB + a 4th service node. Each with position {x, y}, status, and cost field. At least 2 edges connecting them.

Query hooks

- `useApps()` — `useQuery(['apps'], fetchApps)` with loading/error state.
- `useGraph(appId)` — `useQuery(['graph', appId], () => fetchGraph(appId), { enabled: !!appId })`. Automatically refetches when `appId` changes (TanStack Query cache key change).

UI states to show

Loading → Skeleton cards or spinner. Error → Error banner with retry button. Success → render ReactFlow with returned nodes + edges.

---

6. Zustand Store

One store file: `src/store/useAppStore.ts`. Do **not** store server data here — that lives in TanStack Query cache.

State key	Type	Default	What it does
selectedAppId	string | null	null	Currently viewed app — drives graph query
selectedNodeId	string | null	null	Highlighted node — drives inspector panel
isMobilePanelOpen	boolean	false	Controls right-panel drawer on mobile
activeInspectorTab	string	'config'	Which inspector tab is active

Rule: if data can be derived from query cache (e.g. selected node's label), use a selector — do not duplicate it into Zustand.

---

7. TypeScript & Tooling Setup

tsconfig.json must include

- `"strict": true`
- `"noImplicitAny": true`
- `"strictNullChecks": true`
- `"paths": { "@/*": ["./src/*"] }` — for clean imports

Required npm scripts (`package.json`)

Script	Command	Purpose
dev	`"vite"`	Start dev server
build	`"tsc && vite build"`	Type-check then build
preview	`"vite preview"`	Serve production build
lint	`"eslint src --ext .ts,.tsx"`	Run ESLint
typecheck	`"tsc --noEmit"`	Type-check without emitting

ESLint packages to install

```
eslint · @typescript-eslint/parser · @typescript-eslint/eslint-plugin · eslint-plugin-react · eslint-plugin-react-hooks
```

---

8. Step-by-Step Installation Commands

1. Create project

```
npm create vite@latest app-graph-builder -- --template react-ts && cd app-graph-builder
```

2. Install ReactFlow

```
npm install @xyflow/react
```

3. Install TanStack Query

```
npm install @tanstack/react-query @tanstack/react-query-devtools
```

4. Install Zustand

```
npm install zustand
```

5. Install shadcn/ui

```
npx shadcn-ui@latest init (choose Vite + TS when prompted)
```

6. Add shadcn components

```
npx shadcn-ui@latest add button input badge tabs slider textarea
```

7. Install MSW

```
npm install msw --save-dev && npx msw init public/ --save
```

8. Install ESLint + TS plugin

```
npm install -D eslint @typescript-eslint/parser @typescript-eslint/eslint-plugin
eslint-plugin-react eslint-plugin-react-hooks
```

9. Install Tailwind (shadcn peer)

```
npm install -D tailwindcss postcss autoprefixer && npx tailwindcss init -p
```

---

9. Final Submission Checklist

✓	Top Bar with brand title, Fit-view button, placeholder actions
✓	Left Rail with icon-style static navigation
✓	Right Panel: App list (top) + Node Inspector (bottom)
✓	Right Panel becomes a slide-over drawer on mobile (Zustand-controlled)
✓	Dotted ReactFlow canvas (Background variant='dots')
✓	At least 3 nodes + 2 edges rendered on canvas
✓	Nodes are draggable
✓	Clicking a node sets selectedNodeId in Zustand + shows inspector
✓	Delete/Backspace key removes selected node
✓	Zoom and pan works (ReactFlow default)
✓	Fit View button wired to reactFlowInstance.fitView()
✓	Node Inspector: Status badge (Healthy/Degraded/Down)
✓	Node Inspector: Min 2 tabs (Config + Runtime) — activeInspectorTab in Zustand
✓	Node Inspector: Slider (0–100) + numeric input fully synced both ways
✓	Node Inspector: Node name editable input persists to ReactFlow node data
✓	MSW handler: GET /apps with simulated latency
✓	MSW handler: GET /apps/:id/graph with simulated latency + optional error
✓	TanStack Query: useApps() hook with loading + error state in UI

✓	TanStack Query: useGraph(appId) refetches when app changes
✓	Zustand store: selectedAppId, selectedNodeId, isMobilePanelOpen, activeInspectorTab
✓	TypeScript strict mode enabled in tsconfig.json
✓	ESLint configured for React + TypeScript
✓	All required npm scripts: dev, build, preview, lint, typecheck
✓	GitHub repository with source code
✓	README with setup instructions, key decisions, known limitations

10. Bonus Features (only if time permits)

- **Add Node button** — creates a new ServiceNode at a random canvas position.
- **Node types** — Service node vs DB node with different background colours/icons.
- **Persist inspector edits** — slider / name changes write back to ReactFlow `setNodes`.
- **Keyboard shortcuts** — F = fit view, Escape = close inspector / panel.

11. README.md Template

Your README must contain at minimum these three sections:

- **Setup Instructions:** 1. Clone repo 2. npm install 3. npm run dev — list any env vars if needed.
- **Key Decisions:** Why MSW over vite-plugin-mock, why Zustand slice design, why custom node vs default, etc.
- **Known Limitations:** E.g. no real backend, no persistence, mobile drawer tested on Chrome only.

Focus on correctness, clean architecture, and TypeScript strictness over extra features. A well-structured, working core implementation scores higher than an incomplete feature-rich one.